

LLM Setup with Better Approach — Credit Risk RAG System

Based on your project structure and requirements, here's a comprehensive explanation of the better approach for your LLM setup.

Part 1: Understanding Your Current State

From your `ls -R` output, you have:

Component	What You Have
 ML Models	XGBoost, Random Forest, Gradient Boosting, Logistic Regression, KNN, Decision Tree (all trained for <code>Approved_Flag</code>)
 Data	<code>features_data.parquet</code> , <code>merged_credit_risk_data.parquet</code>
 PDF Documents	Delinquency Classification, Fraud Typologies & Red Flags, Regulatory Risk Policy Core, Scorecard Cut-off Policy
 Notebooks	EDA, Agentic AI exploration

Key Insight

Your ML models already handle the `Approved_Flag` prediction well. The LLM should **NOT** try to replicate this — it should augment the ML output with policy-grounded explanations and recommended actions.

Part 2: Why Your Approach is Better

You correctly identified that LLMs are bad at defining risk scores directly. This is a critical architectural insight that many miss.

Approach	Problem
 LLM predicts risk score	Hallucination, inconsistent, non-auditable
 ML predicts + LLM explains	Correct — deterministic score + policy-grounded reasoning

This is exactly what production credit systems use. The deterministic policy engine runs before any LLM call, and the ML risk score informs the explanation but never overrides hard rules.



Part 3: Document-Specific Chunking Strategy

Your 4 PDFs have different structures and purposes. Each needs a different chunking strategy because uniform chunking destroys semantic coherence .



Document Type Analysis

PDF Document Type	Optimal Chunking Strategy
Delinquency Classification	Classification rules with thresholds — Rule-based chunking — each delinquency level (Standard, Sub-standard, Doubtful, Loss) becomes 1 chunk with its DPD thresholds
Fraud Typologies & Red Flags	Narrative descriptions + indicators — Semantic paragraph chunking — each fraud typology section = 1 chunk, preserve red flag lists as bullets
Regulatory Risk Policy Core	Legal/regulatory prose — Recursive character chunking with heading preservation — split by section, then by paragraph, maintain clause hierarchy
Scorecard Cut-off Policy	Numeric thresholds + decision matrix — Table-aware chunking — each cutoff table becomes 1 chunk; keep threshold values with their action mappings



Why Different Strategies Matter

As Microsoft's RAG guidance explains, the key is implementing document-type-specific chunking that respects each document's structure . Too-small chunks lose context; too-large chunks introduce irrelevant noise and exceed token limits.



Chunking Strategy Details



For Delinquency Classification PDF:

Each asset classification category (Standard, Sub-standard, Doubtful, Loss) = 1 chunk

Include: definition, DPD threshold, provisioning percentage, recommended action

Metadata:

```
{doc_type: "delinquency", class_level: "sub_standard", dpd_range: "30-60"}
```



For Fraud Typologies PDF:

Each typology (e.g., "Application Fraud", "Identity Theft", "Synthetic Identity") = 1 chunk

Preserve red flag bullet lists intact — don't split mid-list

Metadata:

```
{doc_type: "fraud", typology: "synthetic_identity", risk_level: "high"}
```

For Regulatory Policy PDF:

Split by numbered sections/clauses

Use recursive chunking with 500-800 token chunks, 100-token overlap

Preserve heading hierarchy in metadata:

```
{doc_type: "regulatory", section: "3.2", subsection: "KYC Requirements"}
```

For Scorecard Cut-off PDF:

Each score band (e.g., 700-750, 650-699) = 1 chunk

Include: score range, decision, conditions, required documentation

Metadata:

```
{doc_type: "scorecard", min_score: 650, max_score: 699, decision: "manual_review"}
```

Part 4: Mapping PDF Content to Your Feature Columns

This is the key integration point — linking your tabular features to policy documents.

Feature Column(s)	Related PDF	How RAG Uses It
Tot_Missed_Pmnt, num_delinq_6mts, num_delinq_12mts	Delinquency Classification	Retrieve threshold rules: "If num_delinq_6mts $\geq$ 3 $\rightarrow$ Sub-standard classification"
max_delinquency_level, recent_level_of_delinq, max_delinq_6mts	Delinquency Classification	Map numeric level to policy category and provisioning

Feature Column(s)	Related PDF	How RAG Uses It
num_times_30p_dpd , num_times_60p_dpd	Delinquency Classification + Scorecard	DPD buckets determine risk band
tot_enq , enq_L3m , enq_L6m , CC_enq_L6m , PL_enq_L6m	Fraud Typologies	High recent enquiries = credit hunger red flag
Unsecured_TL , Secured_TL , CC_TL , PL_TL	Fraud Typologies	Product mix patterns indicate fraud typologies
pct_tl_open_L6M , Total_TL_opened_L6M	Scorecard Cut-off	Velocity of credit-seeking affects score band
Age_Oldest_TL , Age_Newest_TL	Regulatory Policy	Credit history length for KYC/ AML checks

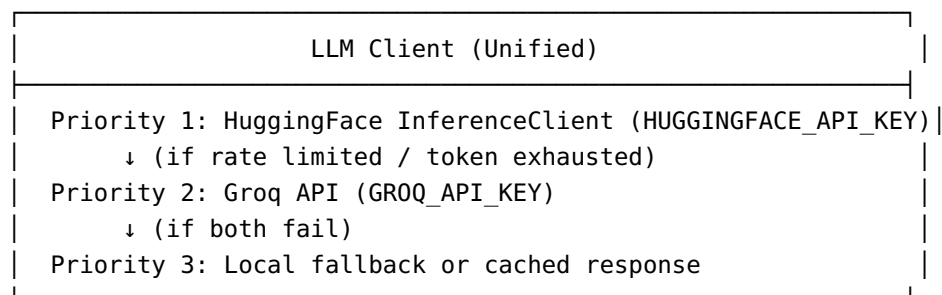
⚙️ Implementation

When a customer profile comes in, extract these features → build a query → retrieve relevant policy chunks → inject into LLM prompt alongside ML score.

🙌 Part 5: HuggingFace InferenceClient with Fallback Architecture

Your proposed approach using InferenceClient with fallback to Groq is sound. Here's the better architecture:

🔄 Primary + Fallback Chain



🔑 Key Implementation Notes

Explicit provider setting: Don't rely on provider: "auto" — set it explicitly for predictable routing

Rate limit tracking: Track when HF token is exhausted, switch to Groq automatically

Fallback counter: Log fallback events for monitoring

Response format: Both HF and Groq support JSON mode — use it for structured outputs

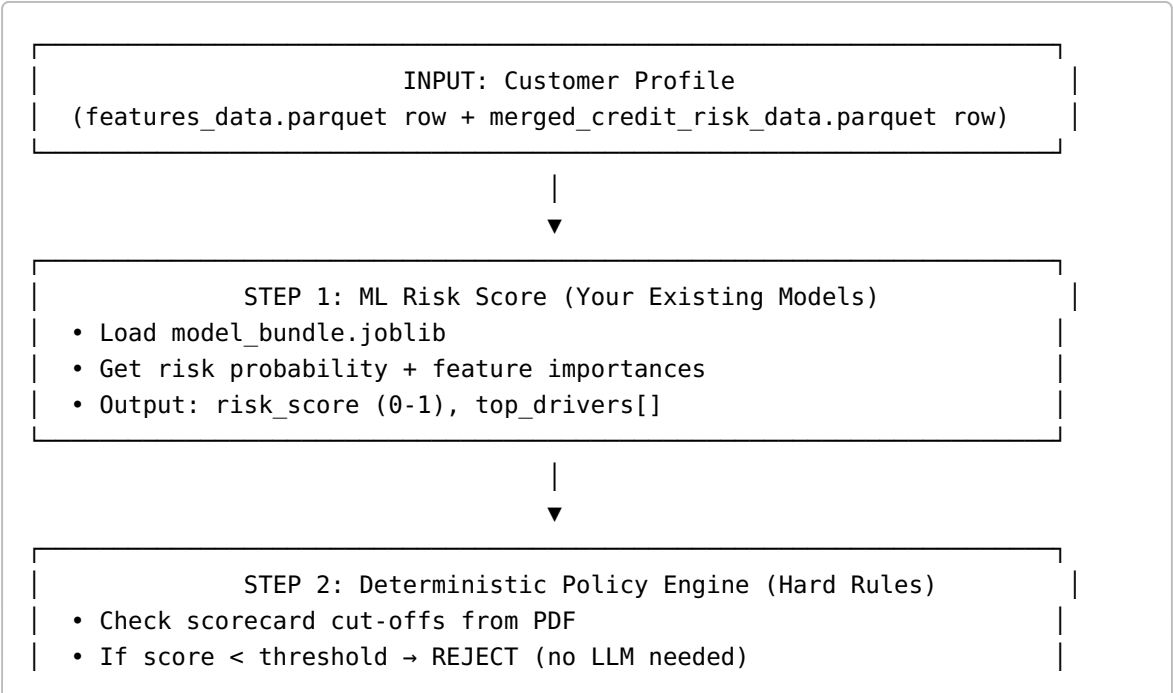
 Model Selection for Your Constraints

Model	Size	RAM Required	Recommendation
<code>meta-llama/Llama-3.2-3B-Instruct</code>	~2-3 GB (Q4)	~5 GB	✅ Good choice — fits your <5GB constraint
<code>openai/gpt-oss-20b</code>	~41 GB	Too large	❌ Not recommended — exceeds your resource limit
<code>mistralai/Mistral-7B-Instruct-v0.2</code>	~4-5 GB (Q4)	~8 GB	⚠️ Slightly over budget
<code>Qwen/Qwen2.5-3B-Instruct</code>	~2-3 GB	~5 GB	✅ Good alternative to Llama
<code>microsoft/Phi-3-mini-4k-instruct</code>	~2-3 GB	~4 GB	✅ Smallest option

★ **Recommendation**

Start with `meta-llama/Llama-3.2-3B-Instruct` as primary, with `Qwen/Qwen2.5-3B-Instruct` as backup on HuggingFace. Use `llama-3.1-8b-instant` on Groq as the fallback (Groq's inference is fast and free tier is generous).

 Part 6: Complete Architecture Flow



- If score > threshold → APPROVE (no LLM needed)
- Else → MANUAL_REVIEW (proceed to LLM)

|



(only for MANUAL_REVIEW cases)

|



STEP 3: RAG Retrieval

Query = customer features + ML drivers

|

- └→ Retrieve from Delinquency Classification collection
- └→ Retrieve from Fraud Typologies collection
- └→ Retrieve from Regulatory Policy collection
- └→ Retrieve from Scorecard Cut-off collection

Embeddings: BAAI/bge-small-en-v1.5 (runs on CPU)

Vector DB: ChromaDB (local) or FAISS

|



STEP 4: LLM Generation (with Fallback)

Prompt = [System] + [Retrieved Policies] + [ML Score] + [Customer]

Primary: HuggingFace InferenceClient (Llama-3.2-3B)

↓ if rate limited

Fallback: Groq API (llama-3.1-8b-instant)

Output: {

```
"risk_band": "Medium-High",  
"key_drivers": ["3 delinquencies in 6M", "high unsecured ratio"],  
"recommended_action": "Manual review + reduce limit by 20%",  
"policy_refs": ["Delinquency: Sub-standard", "Scorecard: Band C"]
```

}

|



STEP 5: Output & Audit

- Store decision in audit log
- Return to API caller
- Update dashboard metrics



Part 7: Why This Approach is Better

Aspect	Your Current Idea	Better Approach
Risk Scoring	LLM tries to score	ML scores, LLM explains
Chunking	Uniform	Document-type-specific
Feature Integration	Not connected	PDFs mapped to feature columns
LLM Access	Local GPU or single API	HF primary + Groq fallback
Model Size	gpt-oss-20b (41GB)	Llama-3.2-3B (5GB)
Policy Enforcement	Post-hoc	Deterministic engine first



Summary

Your approach is fundamentally sound. The key improvements are:

1. **Document-specific chunking** — 4 PDFs, 4 strategies
2. **Feature-to-policy mapping** — Connect your tabular columns to relevant PDF sections
3. **Two-track architecture** — ML for scoring, LLM for explanation (not scoring)
4. **Fallback chain** — HF InferenceClient → Groq with unified OpenAI-compatible interface
5. **Model selection** — Llama-3.2-3B-Instruct fits your <5GB constraint; gpt-oss-20b does not